markpublish User Guide

Modern PDF and HTML publishing from Markdown

> *A practical guide and complete reference for configuring, customising and generating documents with markpublish.*

Author	Frank Winter
Status	Released
Version	1.0.0
Date	2026-09-03
Copyright	© 2026 Frank Winter

Table of Contents

Appendices

CHAPTER 1

Introduction & Architecture

> Goals, core concepts and the modular processing pipeline.

1 Introduction & Architecture

Welcome to the **markpublish User Guide**. `markpublish` is a modern, open-source publishing tool that turns plain Markdown files into professionally typeset **PDF publications** and **HTML previews**.

1.1 Motivation & Goals

Most documentation tools either demand a complex LaTeX setup or stop at plain HTML pages. `markpublish` combines the simplicity of Markdown with the typographic precision of **CSS Paged Media** through the WeasyPrint engine.

1.1.1 Key benefits at a glance:

- **Modular chapters**: every chapter lives in its own Markdown file.
- **Two-level structure**: overarching sections (*parts*) above the chapters; depth within a chapter comes from its own headings.
- **Declarative control**: one manifest (`markpublish.yaml`) governs content, metadata and layout.
- **W3C CSS Paged Media**: exact control over `@page` margins, multi-line running headers and footers, page numbers and divider pages.
- **Extensible pipeline**: a clean separation between parser, renderer and template layers.

1.2 The Processing Pipeline

The architecture of `markpublish` follows a staged process:

1. **Configuration loader (`config.loader`)**: reads the YAML structure, validates data types and resolves dynamic values (for example `date: auto`).
2. **Template resolver (`templates.resolver`)**: finds the theme for the target format along a three-level priority (*user* > *common/workspace* > *package*).
3. **Markdown & TOC engine (`markdown.engine`)**: parses Markdown with extended features (callouts, tables, Pygments syntax highlighting), assigns consistent heading numbers (`1.1`, `1.2`) and builds the tables of contents.
4. **Renderers (`renderers.pdf` & `renderers.html`)**: hand the rendered HTML and CSS to WeasyPrint for PDF export, or write standalone, responsive HTML.

CHAPTER 2

Installation & Quickstart

> Step-by-step installation and your first document.

AT A GLANCE

2 Installation & Quickstart

This chapter shows how to install `markpublish` and compile your first document in under two minutes.

2.1 Requirements

`markpublish` needs **Python 3.10 or newer**. It runs natively on:

- **Windows** (10, 11, Server)
- **macOS** (Intel and Apple Silicon)
- **Linux** (Debian, Ubuntu, Fedora, Arch, …)

2.2 Installing with pip

Install the package with the Python package manager:

```
pip install markpublish
```

For development, or to install from source:

```
git clone https://github.com/fwdotcom/markpublish.git
cd markpublish
pip install -e ".[dev]"
```

2.3 Initialising your first project

Use the `init` command to create a new document:

```
markpublish init my-guide --title "My Guide"
cd my-guide
```

It creates a deliberately minimal stub — two files, directly in the directory:

```
my-guide/
├── markpublish.yaml          # The document manifest
└── next-steps.md             # One chapter, meant to be replaced
```

2.4 Looking things up

```
# two-page reference card
markpublish cheatsheet [--lang de|en]

# this guide
markpublish manual [--lang de|en]
```

Both are rendered from sources shipped inside the package, so they always match the version you have installed. A successful run also confirms that the rendering toolchain works — on Windows, that WeasyPrint found its GTK runtime.

Without `--lang` your system language decides; where no translation exists, the English one appears. The parameter selects the **source**, not merely the interface labels.

2.5 Building the document

Render with the `build` command:

```
# Build a PDF from the current project directory
markpublish build

# Produce the HTML output
markpublish build --target html

# Build both in one pass
markpublish build --target all
```

The finished files are written next to the manifest.

CHAPTER 3

The CLI Commands

> Every command in detail: build, init, cheatsheet, manual, templates, export-template and labels.

AT A GLANCE

3 The CLI Commands

`markpublish` offers a small, predictable command-line interface, reachable as `markpublish` or the short form `mpub`.

3.1 Command overview

Command	Summary
`markpublish init`	Creates a minimal project stub (two files)
`markpublish cheatsheet`	Renders the two-page reference card
`markpublish manual`	Renders this user guide
`markpublish build`	Compiles a document to PDF and/or HTML
`markpublish templates`	Lists every template found and where it came from
`markpublish export-template`	Copies a template out for customisation
`markpublish labels`	Shows the resolved static texts and their origin

3.2 `markpublish init`

Creates a minimal project stub: a `markpublish.yaml` and one chapter, `next-steps.md`, both directly in the target directory. No `chapters/` folder, no sample chapters.

```
markpublish init [DIRECTORY] [--title "Title"]
```

The stub is deliberately small, because it exists to be deleted — at the latest when you write your first real chapter. The reference therefore does not live in the stub but in `markpublish cheatsheet`; both generated files point at it. Cover page and table of contents are switched off, because a one-page draft needs neither.

3.3 `markpublish cheatsheet`

Renders the bundled reference card: page 1 the keys of `markpublish.yaml`, page 2 the essentials of themes and templates.

```
markpublish cheatsheet [--lang CODE] [--target pdf|html|all] [--output PATH] [--theme NAME]
```

3.4 `markpublish manual`

Renders this guide.

```
markpublish manual [--lang CODE] [--target pdf|html|all] [--output PATH] [--theme NAME]
```

3.4.1 Shared behaviour of the two document commands

Both sources ship inside the package and are rendered on every call. The result therefore always matches the version you have installed, and a successful run doubles as proof that the rendering toolchain works — on Windows, that WeasyPrint found its GTK runtime. Only the finished document is written, into the current directory; no source files ever land in your project.

Option	Default	Notes
`--lang`, `-l`	system language	Which translation to render. `markpublish manual --lang de`
`--target`, `-t`	`pdf`	`pdf`, `html` or `all`
`--output`, `-o`	current directory	File or directory
`--theme`	built-in	Render with a theme of your own

`--lang` selects the **source**, not just the labels — each translation carries its own `language:` and therefore pulls in the matching static texts by itself.

Given no language, your system decides. These two documents address the person at the keyboard rather than an audience, so their language is the best guess available. Where no translation exists, the English one appears without comment — nothing was asked for. An explicit `--lang fr`, by contrast, is reported, rather than silently wrapping an English frame around a French expectation.

Without `--theme` or `--templates-dir`, both commands deliberately ignore user and project themes and render with the built-in one. Otherwise a `templates/` folder in your working directory — created with `export-template` and still half-finished — would drag the reference down with it: one missing label would abort the build at exactly the moment somebody wants to look up how labels work.

If the PDF path is unavailable for want of GTK, `--target html` produces the same document without WeasyPrint.

3.5 `markpublish build`

Compiles a `markpublish.yaml` configuration.

```
markpublish build [CONFIG_FILE] [OPTIONS]
```

3.5.1 Arguments & options

- `CONFIG_FILE` *(optional, default `markpublish.yaml`)*: path to the YAML file.

- `--target / -t` *(default `pdf`)*: target format (`pdf`, `html` or `all`).
- `--output / -o`: custom output path (file or directory).
- `--templates-dir`: explicit path to a shared templates directory.

3.5.2 Examples

```
# Default build from the current directory
markpublish build

# Write the HTML version into a specific directory
markpublish build markpublish.yaml -t html -o dist/

# Use a shared templates directory
markpublish build --templates-dir /shared/company-templates
```

> **Note**
>
> There is no `--lang` on `build`. A document's language belongs in its own `markpublish.yaml`, next to `title` and `author`: it states which language the document is *written in*. Overriding it from the command line would only swap the seventeen static labels around unchanged prose.

3.6 `markpublish templates`

Shows every template on the system in a table, together with its priority:

```
markpublish templates
markpublish templates --target pdf
```

3.7 `markpublish export-template`

Copies the built-in theme into your working directory:

```
markpublish export-template default templates
```

3.8 `markpublish labels`

Shows which static text applies in the end, and which level of the label cascade it came from. Useful as soon as a theme brings its own texts and it stops being obvious which level wins.

```
markpublish labels                      # all keys, target PDF
markpublish labels --target html        # cascade for the HTML output
```

```
markpublish labels --overridden            # only overridden texts
markpublish labels path/to/markpublish.yaml
```

3.8.1 Arguments & options

Parameter	Type	Default	Description
`config_file`	Path	`markpublish.yaml`	Path to the configuration file
`--target`, `-t`	String	`pdf`	Target format whose cascade is shown (`pdf` or `html`)
`--templates-dir`	Path	–	Alternative templates directory
`--overridden`	Flag	off	Only show texts the theme changes

The *Source* column names the level: `i18n.yaml (en)` for the program default, or the path of a theme or target `i18n.yaml`. Below the table, markpublish lists which files were searched for overrides and which of them exist.

> **Tip**
>
> `--overridden` answers the most common question directly: *what in this project deviates from the default at all?*

CHAPTER 4

Configuration Guide

> Declarative definition of document metadata, chapters and hierarchies.

AT A GLANCE

4 Configuration Guide

`markpublish.yaml` is the control document of every publication. It falls into three parts: `document` for metadata and global switches, `theme` for choosing the design, and `chapters` for the structure of the document.

4.1 The `document` object

Everything global — metadata and layout switches — lives under `document:`:

```yaml
document:
  title: "markpublish User Guide"
  subtitle: "Modern PDF and HTML publishing"
  summary: "Short abstract for the cover page and metadata."
  author: "Frank Winter"
  organisation: "WASDCAT Games"
  date: "auto"                # "auto" (today) or a fixed date "2026-08-31"
  version: "1.0.0"            # optional - omit it and the field disappears
  language: "en"              # labels and date format; defaults to the system language

  # Global switches
  cover: true                 # Cover page on/off
  document_toc: 2             # Large TOC, two levels deep
  autonum_style: "decimal"    # "decimal", "roman", "legal", "none"
  chapter_toc: 2              # Default for the chapter divider pages
  header: true                # Running header
  footer: true                # Running footer
```

4.1.1 Automatic date resolution

With `date: "auto"` or `date: "today"`, markpublish inserts the current date in the format matching the document language:

- `language: "de"` → `31.08.2026`
- `language: "en"` → `2026-08-31`

4.1.2 Additional metadata fields

Beyond the keys listed above, `document:` accepts **any field you like**. Each one appears in the metadata grid on the cover page without further work:

```yaml
document:
  title: "Project Report"
  department: "IT Architecture"
  client: "Example Ltd."
  approved: true
```

Three things are worth knowing:

- **The caption comes from the i18n cascade.** With no entry for the key, the cover prints the key itself — `department` rather than `Department`. Put an `i18n.yaml` next to your `markpublish.yaml` and name it there:

```yaml
# i18n.yaml, in the project directory
en:
  department: "Department"
  client: "Client"
  approved: "Approved"
```

- **The type survives.** A `true` stays a boolean rather than becoming text: the cover prints `Yes` or `Ja`, following the document language. Both words live in the cascade as `bool_true` and `bool_false`.
- **A typo is printed, not reported.** `titel:` instead of `title:` produces no error — it produces an extra line on the cover. `markpublish labels` lists every field with its diagnosis; a look there before the first build saves the search.

Themes read these fields from the `meta` dictionary described in *The Template and Design System*.

4.2 Defining a chapter

A plain chapter is given as `file:`, with optional `title:` and `summary:`:

```yaml
parts:
  - title: "Main"
    break_before: "none"
    chapters:
      - file: "chapters/01_intro.md"
        title: "Introduction"
        summary: "What this guide sets out to do."
        break_before: "divider"    # Its own divider page ahead of the chapter
        chapter_toc: "none"        # No per-chapter mini TOC
```

A chapter always sits under a part — a `chapters:` at the top level is rejected with a pointer to the right shape. The next section explains why.

All paths are relative to `markpublish.yaml`, not to the shell's working directory.

4.3 Document Structure in 2 Tiers: Parts and Chapters

`markpublish` organises documents in a clean 2-tier structure:

1. **Tier 1 — Parts (`parts:`):** Organises the document into major logical sections (e.g. Main Body, Appendices, Volumes). Every part must have a name (`title:` or `part:`).
2. **Tier 2 — Chapters (`chapters:`):** The actual content files (*.md) belonging to each part.
3. **Internal Structure (`##`, `###`):** Formatted directly in Markdown.

```yaml
parts:
  - title: "Main"              # Main section
    break_before: "none"       # No part divider page for the main part
```

```yaml
    document_toc: "none"       # 'Main' is not listed in TOC; chapters appear directly
    chapters:
      - file: "chapters/01_intro.md"
        title: "Introduction"
      - file: "chapters/02_usage.md"
        title: "Usage"

  - title: "Appendices"        # Section with divider page & TOC rubric
    summary: "Supplementary tables and references."
    break_before: "divider"
    autonum_from_level: 2
    document_toc: 2
    chapters:
      - file: "chapters/appendix_a.md"
        title: "Appendix A: Reference"
        autonum_prefix: "A."
      - file: "chapters/appendix_b.md"
        title: "Appendix B: Glossary"
        autonum_prefix: "B."
```

A named part groups its chapters, optionally receives a divider page, and appears as a structural category in the table of contents (unless omitted via `document_toc: "none"`). All chapters align to the same primary baseline.

CHAPTER 5

The Template and Design System

> Three-level resolution, CSS Paged Media and multi-line running headers.

AT A GLANCE

5 The Template and Design System

The template system of `markpublish` allows complete visual customisation while staying maintainable.

5.1 Theme-based directory structure

Templates are grouped by theme, with the output format underneath. Everything belonging to one design therefore stays together, and a theme can be copied, shared or versioned as a whole:

```
templates/
└── default/                      # Theme name
    ├── pdf/
    │   ├── layout.html           # HTML skeleton
    │   ├── styles.css            # CSS Paged Media, header and footer
    │   ├── fonts/                # Bundled font (Open Sans, variable) + OFL.txt
    │   ├── cover.html            # Cover page template
    │   ├── part_divider.html     # Divider page for overarching parts
    │   ├── chapter_divider.html  # Chapter divider page with mini TOC
    │   └── toc.html              # Global table of contents
    └── html/
        ├── layout.html           # Standalone web layout
        ├── styles.css            # Screen/responsive stylesheet
        └── ...
```

5.2 The three-level resolution hierarchy

When looking for a template (say `default/pdf`), this strict priority applies:

```
1. User directory (~/.markpublish/templates/default/pdf/)
   │ (highest priority)
   ▼
2. Common / project folder (<templates_dir>/default/pdf/)
   │
   ▼
3. Package built-in (inside the markpublish Python package)
```

5.2.1 Configuring the common templates folder

There are four ways to set the shared template path:

1. **CLI parameter**: `markpublish build --templates-dir /path/to/templates`
2. **In `markpublish.yaml`**: `templates_dir: "./custom-templates"`
3. **Environment variable**: `MARKPUBLISH_TEMPLATES_DIR=/path/to/templates`
4. **Automatic fallback**: `./templates` in the project folder.

5.3 Static texts and language

The fixed labels — table-of-contents heading, chapter marks, cover labels, page-number footer, callout titles — are not in the template but in `i18n.yaml` files. Which language applies is decided by `document.language`:

```yaml
document:
  title: "User Guide"
  language: "en"      # governs labels, callout titles and date format
```

`de` and `en` ship with markpublish. Regional forms are mapped (`de-AT` to `de`), and an unknown language falls back to English. Leave `language:` out altogether and the system language applies — `markpublish init` writes it down straight away, so that a document builds the same on every machine.

> **Note**
>
> **i18n** refers to the sources — the files and the YAML entry, across all languages. **labels** is the result resolved from them for *one* document in *one* language: what arrives in the template as `{{ labels.chapter }}`.

5.3.1 The three-level i18n cascade

All three levels are built **identically**: language code at the top, texts underneath. Each deeper level overrides the one above it — and only **the keys it actually sets**. Everything else stays as it stands further up:

```
1. Program      markpublish/i18n.yaml
   │            complete, de + en
   ▼
2. Theme        <templates>/<theme>/i18n.yaml
   │            applies to every target format of the theme
   ▼
3. Target       <templates>/<theme>/<target>/i18n.yaml
                PDF only, or HTML only
```

Level 1 is the only one that must be complete. It additionally places English beneath the document language, so that every program text is guaranteed to resolve. Levels 2 and 3 are pure overrides.

Static texts are defined **in the theme only**. There is no document level: `document.i18n` in `markpublish.yaml` is rejected, with a pointer to the theme file. To change the texts of a document, give it its own theme — `markpublish export-template` creates one.

> **Important**
>
> Levels 2 and 3 always come from **exactly one** theme: which one applies is decided beforehand by template resolution (user > project > package). A project theme does **not** inherit the texts of the package theme of the same name — only that one theme is merged with the program default.

> **Important**
>
> Levels 2 and 3 apply **only to the chosen language**. A theme that defines an `en:` block does not change a German output — otherwise English theme texts would bleed into documents written in other languages.

5.3.2 The shared format

```yaml
# templates/mytheme/i18n.yaml
de:
  part: "Abschnitt"
  chapter_toc_title: "Auf dieser Seite"
en:
  part: "Section"
  chapter_toc_title: "On this page"
```

The special key `"*"` applies to **every** language and takes effect before the language-specific block — for terms that should read the same regardless of the document language:

```yaml
"*":
  version: "Rev."       # "Rev." in every language
de:
  part: "Abschnitt"
```

If you maintain only one language, the language level may be omitted. The flat form means the same as putting the keys under `"*"`:

```yaml
part: "Section"
```

Regional blocks beat the base block: with `language: "de-AT"`, `de-at:` wins over `de:`. A missing `i18n.yaml` is not an error — a theme without its own texts is the normal case. If one exists but is malformed, the build aborts naming the file, rather than quietly falling back to the defaults.

5.3.3 Example: labelling PDF and HTML differently

```
templates/mytheme/
├── i18n.yaml           en: chapter: "Chapter"
├── pdf/
│   └── i18n.yaml       en: chapter: "Ch."       (PDF only)
└── html/
    └── i18n.yaml       (empty, inherits "Chapter")
```

The bundled `default` theme ships all three files as a **commented-out pattern**: `templates/default/i18n.yaml`, `default/pdf/i18n.yaml` and `default/html/i18n.yaml`. They are deliberately inert — the default theme should read exactly like the program default. Uncomment what you want to change.

`markpublish export-template` copies these files along, including the theme-level `i18n.yaml`, which sits next to the target folders rather than inside them.

5.3.4 Free labels: a template's own texts

A theme may define **its own keys**, which the program does not know — for the static texts of the template itself. The structure is the same, and so is the access:

```yaml
# templates/mytheme/i18n.yaml
de:
  imprint_title: "Impressum"
  disclaimer: "Alle Angaben ohne Gewähr."
en:
  imprint_title: "Imprint"
  disclaimer: "All information without guarantee."
```

```jinja
<!-- templates/mytheme/html/layout.html -->
<footer>{{ labels.imprint_title }}</footer>
```

> **Warning**
>
> For free labels there is **no program default** that could step in. A strict rule therefore applies: a label a template writes down **must** resolve somewhere in the cascade. If it does not, the build aborts and names the key, the location with line number, the document language and the files searched:
>
> ```
> Label 'imprint_title' is used in the template but defined in no i18n level.
>   Document language: en
>   Found at:
>     templates/mytheme/html/layout.html:108
>   Searched in:
>     templates/mytheme/html/i18n.yaml  (present)
>     templates/mytheme/i18n.yaml       (present)
>     markpublish/i18n.yaml             (present)
> ```
>
> So maintain every language you ship, or put the key under `"*"`. An empty text in a finished PDF goes unnoticed — an abort does not.

What is checked are the template sources, not the render pass: a label in a branch this particular document never traverses is reported too. `styles.css` counts as well, since it runs through the same Jinja environment.

5.3.5 Inspecting the resolved table

With three levels it is not always obvious where a text comes from. `markpublish labels` shows the result together with its origin:

```
markpublish labels                    # all keys, target PDF
markpublish labels --target html      # cascade for the HTML output
markpublish labels --overridden       # only what comes from the theme
```

5.3.6 Available keys

Key	de	en
`toc_title`	Inhaltsverzeichnis	Table of Contents
`toc_sidebar`	Inhalt	Contents
`chapter_toc_title`	Inhalt dieses Kapitels	In this chapter
`chapter`	Kapitel	Chapter
`part`	Teil	Part
`author`	Autor	Author
`status`	Status	Status
`version`	Version	Version
`date`	Datum	Date
`copyright`	Copyright	Copyright
`page`	Seite	Page
`page_of`	von	of
`alert_note`	Hinweis	Note
`alert_tip`	Tipp	Tip
`alert_important`	Wichtig	Important
`alert_warning`	Warnung	Warning
`alert_caution`	Achtung	Caution

The `alert_*` titles are produced while parsing Markdown, not later in the template — the cascade is handed through to that point, so an `alert_note` set in the theme takes effect in the callout as well.

To add a new language, complete a block in `markpublish/i18n.yaml` — or, without touching the package, set every key under the desired language code on level 2 or 3.

> **Tip**
>
> In your own templates you reach the result with `{{ labels.chapter }}` — in `styles.css` too, which runs through the same Jinja environment. That is how the footer is built: `"{{ labels.page }} " counter(page) " {{ labels.page_of }} " counter(pages)`.

5.4 Running headers and footers

Header and footer are **running elements**: a block in `layout.html` is given `position: running(name)`, which takes it out of the text flow; the `@page` rule then places it into the margin box with `content: element(name)`.

The difference from a `content:` string matters: the margin box thereby holds **real markup**. Any number of lines becomes possible, each with its own formatting — a string knows only one formatting for everything.

```
<!-- layout.html -->
<div class="page-header-runner">
  <div class="hf-left">
    <div class="hf-doc-title">{{ document.title }}</div>
    <div class="hf-doc-subtitle">{{ document.subtitle }}</div>
  </div>
  <div class="hf-right"><span class="hf-section"></span></div>
</div>
```

```
/* styles.css */
.page-header-runner {
  position: running(pageheader);
  display: flex;
  justify-content: space-between;
  align-items: flex-start;    /* right column stays at the top */
}
.hf-doc-title { font-weight: 700; }
.hf-section::before { content: string(current-section); }

@page {
  @top-left {
    content: element(pageheader);
    width: 100%;
    vertical-align: top;
    margin-top: 12mm;          /* distance to the paper edge */
    padding-bottom: 0;
    border-bottom: 0.5pt solid #cbd5e1;
    margin-bottom: 12mm;       /* distance to the content */
  }
}
```

5.4.1 Geometry

From the paper edge inwards:

```
Edge -- margin -- lines (top-aligned) -- rule -- margin -- content
        12 mm                                    12 mm
```

Both distances are deliberately equal: a header sitting tightly above the text reads as part of the type area rather than as page furniture.

Both columns sit in a flex container with `align-items: flex-start`. The right column therefore begins at the height of the **first** left-hand line, even when there are three lines on the left and only one on the right.

> **Important**
>
> The distance to the content comes from `margin-bottom`, not `padding-bottom`. In CSS the border sits **outside** the padding — a `padding-bottom` would push the rule away from the text and press it against the content. The opposite is wanted: rule right at the text, space after it.

> **Warning**
>
> A margin box **does not push the content**. Its height is capped by the page margin; extra lines run off the page instead of shrinking the type area. The page margin is therefore computed in `styles.css` from the number of lines:
>
> ```
> margin-top = edge distance + lines x line height + rule width + content distance
> ```
>
> If you add a line in `layout.html`, adjust `hf_header_lines` or `hf_footer_lines` in `styles.css` accordingly.

5.4.2 What the default theme puts there

```
Header      Document title (bold)                      Chapter title
            Subtitle

Footer      Copyright                   Version 1.0.0 | 2026-09-01
                                                  Page X of Y
```

Subtitle and version are optional. Without a subtitle the header has a single line and the type area moves up accordingly. Without a version it disappears together with its separator — the footer then shows only the date, and the field is absent from the cover page entirely.

The chapter title comes from `string(current-section)`, which the chapter `<article>` sets via `string-set`; `counter(page)` works inside the running element just as it does in a margin box. The switches `document.header` and `document.footer` hide the respective block; on cover and divider pages it is switched off anyway.

CHAPTER 6

Markdown Features & Syntax

> Tables, admonitions and callouts, Pygments highlighting and task lists.

6 Markdown Features & Syntax

`markpublish` ships a capable Markdown engine with full support for modern documentation elements.

6.1 Code blocks & syntax highlighting

Source code is highlighted by Pygments:

```
from markpublish.config.loader import load_config
from markpublish.markdown.engine import MarkdownPipeline

# Load the manifest
config = load_config("markpublish.yaml")
print(f"Building: {config.document.title}")
```

6.2 Tables

Tables are rendered with a clean zebra pattern and protection against page breaks:

Parameter	Type	Default	Description
`title`	String	*required*	Main title of the document
`author`	String	`None`	Author name
`status`	String	`None`	Document status (e.g. “Released”, “Draft”)
`copyright`	String	`None`	Copyright notice (e.g. “© 2026 Frank Winter”)
`version`	String	`null`	Version identifier. Without it the field disappears from the cover page; when set, it appears in the footer to the left of the date

6.3 GitHub-style callouts (alerts)

`markpublish` supports the common **GitHub alert syntax** with five semantic types. The icons come straight from the template:

> **Note**
>
> An informative note with a blue accent and a matching info icon.

> **Tip**
>
> Use `markpublish build --target all` to produce PDF and HTML in one pass.

> **Important**
>
> Callout titles are localised automatically from the document language (*Hinweis*, *Tipp*, *Wichtig*, *Warnung*, *Achtung* in German).

> **Warning**
>
> With relative image paths, make sure they are relative to the Markdown file that references them.

> **Caution**
>
> Malformed YAML indentation aborts the build while parsing the manifest.

> **A title of your own**
>
> After the alert type you may give an individual title, which replaces the default label.

The classic `!!! note` syntax remains supported as well.

6.4 Task lists

- [x] Initialise the project
- [x] Write the chapters
- [ ] Publish the document

SECTION

Appendices

Complete specification tables, troubleshooting and best practices.

CONTENTS OF THIS SECTION

Appendix A: YAML Schema Reference

A complete overview of every configuration option in `markpublish.yaml`.

A.1 Document properties (`document`)

Key	Type	Default	Description
`title`	String	*required*	Title of the publication
`subtitle`	String	`null`	Subtitle
`summary`	String	`null`	Abstract for the cover page
`author`	String	`null`	Author name
`status`	String	`null`	Document status (e.g. “Draft”, “Released”)
`copyright`	String	`null`	Copyright notice (e.g. “© 2026 Frank Winter”)
`date`	String	`"auto"`	A date, or `"auto"` for today
`version`	String	`null`	Version identifier
`language`	String	system language	ISO language code (`de`, `en`, …)
`cover`	Bool	`true`	Enable the cover page
`document_toc`	String/Int	`full`	Default for the document TOC
`part_toc`	String/Int	`full`	Default for part divider TOCs
`chapter_toc`	String/Int	`full`	Default for chapter divider TOCs
`autonum_style`	String	`"decimal"`	Default numbering style (`decimal`, `roman`, `legal`, `none`)
`autonum_from_level`	Int	`1`	Default start heading level for numbering
`autonum_prefix`	String	`null`	Default prefix for numbers
`autonum_reset`	Bool	`false`	Default reset flag
`pagenum_reset`	Bool	`false`	Reset page number counter
`header`	Bool	`true`	Enable running header
`footer`	Bool	`true`	Enable running footer

A.2 Part properties (`parts`)

Key	Type	Default	Description
`title` / `part`	String	*required*	Part title (e.g. `"Appendices"`, `"Main"`)
`subtitle`	String	`null`	Subtitle for the part divider page
`summary`	String	`null`	Abstract for the part divider page
`break_before`	String	`"divider"`	`"divider"`, `"page"` or `"none"`
`document_toc`	String/Int	`full`	Part contribution to the document TOC
`part_toc`	String/Int	`full`	Local TOC on the part divider page
`chapter_toc`	String/Int	`full`	Default chapter TOC for chapters in this part
`autonum_style`	String	`"decimal"`	Numbering style for this part
`autonum_from_level`	Int	`1`	Start heading level for numbering
`autonum_prefix`	String	`null`	Optional prefix for generated numbers (e.g. `"A."`)
`autonum_reset`	Bool	`false`	Reset counter at part/chapter start
`pagenum_reset`	Bool	`false`	Reset page numbering back to 1 at part start
`chapters`	List	`[]`	Flat list of chapters in this part

A.3 Chapter properties (`chapters`)

Key	Type	Default	Description
`file`	String	`null`	Path to the Markdown file
`title`	String	`null`	Overrides the title taken from the file
`subtitle`	String	`null`	Subtitle for the chapter divider page
`summary`	String	`null`	Abstract for the divider page
`break_before`	String	`page`	How the chapter is set off: `page`, `divider` or `none`
`document_toc`	String/Int	`full`	Chapter contribution to the front TOC
`chapter_toc`	String/Int	`full`	Local TOC on the chapter divider page
`autonum_style`	String	`"decimal"`	Numbering style for this chapter
`autonum_from_level`	Int	`1`	Start heading level for numbering

Key	Type	Default	Description
`autonum_prefix`	String	`null`	Optional prefix for generated numbers (e.g. `"A."` → `A.1`, `A.2`)
`autonum_reset`	Bool	`false`	Reset counter at chapter start
`pagenum_reset`	Bool	`false`	Reset page numbering back to 1 at chapter start

A.3.1 `break_before` — how a chapter is set off

One key with three values, not two independent switches:

Value	Effect
`page`	Default. The chapter starts at the top of a new page
`divider`	A divider page of its own precedes the chapter
`none`	The chapter runs on in the text flow

That it is *one* key has a reason: the three values are one axis, not three questions. As two booleans you could write down “divider page, but no page break” — a state that does not exist, because a divider page always breaks. A setting you can write down and that does nothing is a source of error with no upside.

`page` is what a typeset document is expected to do. If you want short sections to run on — leaflets, reference cards, tightly set appendices — put `break_before: "none"` on the individual chapter.

With `divider`, the chapter’s own break and the divider page’s break fall in the same place and merge into one — no blank sheet appears. Before the first chapter the rule does not apply.

An unknown value aborts the build instead of quietly falling back to `page`: a mistyped `divder` would otherwise become an ordinary page break without complaint, and you would find the missing divider page only when leafing through the finished PDF.

A.3.2 `chapter_toc` and `document_toc` — two tables of contents, one vocabulary

A document has two tables of contents, and each has a pair of keys — one for the default, one for the individual case:

Table of contents	Default under `document:`	On a chapter
The **large** one at the front	`document_toc`	`document_toc`
The **small** one on a divider page	`chapter_toc`	`chapter_toc`

All four take the same three forms:

Value	Meaning
`none`	Does not appear in this table of contents at all
`full`	Every level
number	Down to this depth, counted from the chapter heading

Depth counts **within the chapter**: 1 is the chapter heading itself, 2 the level below it. `document_toc: 1` therefore puts the chapter into the large TOC but none of its sub-headings. `chapter_toc: 2` lists exactly the level below the chapter heading on the divider page.

For the common case, two lines in the `document` block are enough:

```yaml
document:
  document_toc: 2       # large TOC, two levels deep
  chapter_toc: 2        # small ones likewise
```

`document_toc: "none"` leaves the large table out entirely; the per-chapter settings are then moot.

A.3.3 Inheritance

`document_toc` is inherited downwards: set on a part, it applies to every chapter below it, and a chapter passes it on to its sub-chapters. For the most common case — an appendix whose internal structure only bloats the table of contents — one line is enough:

```yaml
  - part: "Appendices"
    document_toc: 1
    chapters:
      - file: "chapters/07_appendix_yaml_spec.md"
        title: "Appendix A: YAML Schema Reference"
      - file: "chapters/08_appendix_troubleshooting.md"
        title: "Appendix B: Troubleshooting"
```

An individual chapter beats what it inherited — including back to `full`. That is exactly why the keyword exists: without it you would have to write an arbitrarily large number to say “everything after all”.

`chapter_toc` is **not** passed from chapter to sub-chapter. It describes the divider page of this one chapter, and every chapter has its own; without a value of its own, `document.chapter_toc` simply applies.

A.3.4 What is shortened, and what is not

The prose is untouched: the sub-headings remain in the chapter, numbering and anchors included. Only the table of contents is shortened. A sub-chapter keeps its own entry — the depth is inherited relative to each chapter, not applied across the combined list.

An unknown value aborts the build. Booleans are rejected too: a `true` would not show the depth — that is exactly what `full` is for.

A.4 Numbering

`document.autonum_style` sets the style for the whole document; `autonum_style` on a chapter or part departs from it and **passes the value down**. Without that inheritance the setting on a part would do nothing: the headings live in the chapter files, not in the part.

`autonum_style: "none"` means **nothing** in that branch carries a number — neither the chapter heading nor the levels below it. There is therefore no restart at 1 either: no count is running inside the branch that could begin again.

The document counter is left untouched. An unnumbered stretch consumes no number:

```yaml
parts:
  - chapters:
      - file: "chapters/01.md"          # 1
  - part: "Appendices"
    autonum_style: "none"           # appendices: no numbers
    chapters:
      - file: "chapters/a.md"
      - file: "chapters/b.md"
  - chapters:
      - file: "chapters/02.md"          # 2, not 4
```

A.4.1 Numbering from Sub-Headings (`autonum_from_level` & `autonum_prefix`)

For extensive appendices or specialized sections whose main chapter title should remain unnumbered (e.g. *"Appendix A: Reference"*), but whose sub-sections need hierarchical numbering:

- `autonum_from_level: 2` leaves the `h1` chapter heading unnumbered and starts numbering at `h2` (`1`, `2`, ...) and `h3` (`1.1`, `1.2`).
- When `autonum_from_level > 1`, each chapter automatically resets its counter to start afresh.
- `autonum_prefix: "A."` prepends a custom prefix to generated numbers (`A.1`, `A.2`, `A.2.1`).

```yaml
parts:
  - part: "Appendices"
    autonum_from_level: 2           # inherits autonum_style from document
    chapters:
      - file: "chapters/appendix_a.md"
        title: "Appendix A: Reference"
        autonum_prefix: "A."        # A.1, A.2, A.2.1

      - file: "chapters/appendix_b.md"
        title: "Appendix B: FAQ"
        autonum_prefix: "B."        # B.1, B.2, B.2.1
```

Appendix B: Troubleshooting & FAQ

Common questions, error messages, and fixes when producing documents.

Issue / Error Message	Possible Cause	Solution
`cannot load library 'libgobject-2.0-0'`	WeasyPrint requires native Pango and GTK C-libraries on Windows.	Install GTK3 runtime (GTK-Installer) or MSYS2: `pacman -S mingw-w64-x86_64-pango`. Quick alternative: `--target html` renders without WeasyPrint.
Missing graphics / empty image frames	Image path is broken or absolute instead of relative.	Always specify paths relative to the referring Markdown file (e.g. `images/diag.png`). markpublish inlines graphics up to 12 MB as data URIs.
Table of Contents shows wrong page numbers	Manually specified heading IDs collide during two-pass rendering.	Rely on markpublish's automatic slug generator or check custom anchors (`{#id}`).
`Label 'x' is used in template but defined in no i18n level`	A theme uses a static text key not defined in any `i18n.yaml` level.	Define the key in `<theme>/i18n.yaml` for all languages or under `*`. Inspect active labels with `markpublish labels`.
`cheatsheet` or `manual` ignores working directory theme	Built-in reference documents render with the stable default theme by default.	Use `--theme <name>` to explicitly enforce rendering in your custom theme.